

CodeLogic

Problem Statement: Personal Loan Application System

In today's fast-paced digital world, financial institutions are increasingly adopting technology to streamline and simplify the loan management process for both customers and administrative staff. However, many systems lack a comprehensive, user-friendly platform that efficiently handles the entire personal loan lifecycle — from customer onboarding and loan application to EMI calculations and customer support.

Objectives:

To design and develop a **modular, full-stack Personal Loan Application System** that enables:

- Customers can register, apply for personal loans, track application status, calculate EMIs, and raise support tickets.
- Admin to manage loan applications, review and approve/reject requests, handle support tickets, and monitor system statistics through dashboards.

The following are the modules categorised for the above problem statement.

Module 1: User Management and Authentication

Overview

This module provides secure user registration, login, and role-based access control for customers and admins. It uses Spring Boot for backend APIs, JWT for authentication, and Vue.js for frontend forms and routing.

Backend

Entities & Repositories :

- User entity: Contains id, username, password, and role fields.
- UserRepository: Extends JpaRepository for CRUD operations.

APIs :

- POST /register: Registers a new user, hashes the password, assigns default role (CUSTOMER), and saves to the database.
- POST /login: Authenticates user credentials, generates a JWT token, and returns it.

Authentication:

- Uses JWT for stateless authentication.
- Secures endpoints based on user roles.

Frontend

- Registration form: Collects user details and submits to /register.
- Login form: Authenticates user and stores JWT token.
- Routes users to different dashboards based on their role (CUSTOMER or ADMIN).
- User can view their profile

Unit Testing

- JUnit tests for registration and login logic.
- Tests cover password hashing, JWT generation, and validation.

Module 2 : Loan Application Submission and Status Tracking

Overview

This module allows users to submit loan applications and track their status. It enforces business rules for validation and provides RESTful APIs for application management.

Database Table:

loan_applications: Stores loan application details (id, user_id, amount, tenure_months, income, credit_score, status, application_date, purpose).

APIs:

POST /loans/apply: Submits a new loan application.

- Validates income ($> ₹25,000$), credit score (300–900), and checks for duplicate requests within 24 hours.
- Sets initial status to NEW.

GET /loans/user/{userId}: Retrieves all loan applications for a user.

GET /loans/{loanId}: Returns detailed status and information for a specific loan application.

Business Rules:

- Income must be above ₹25,000.
- Credit score must be between 300 and 900.
- Duplicate applications within 24 hours are rejected.

Frontend:

- Customers can apply for the loans which includes fields for amount, income, tenure, credit score, and purpose.
- It is a three step process which include
 1. Loan Details : It takes the user inputs loan amount, tenure and purpose
 2. Financial Info : It takes the user inputs monthly income and credit score
 3. Review : It shows the personal information, financial information and loan information entered till now

Unit Testing

- JUnit tests for REST endpoints and business logic.
- Tests cover input validation, duplicate checks, and status retrieval.

Module 3 : Admin Dashboard and Loan Processing Engine

Overview

This module empowers administrators to manage loan applications efficiently. Admins can view all pending applications, approve or reject them, and add review remarks. The backend enforces business rules for status transitions and records review history for auditability. The frontend provides a dashboard for filtering and processing applications, ensuring a streamlined workflow for loan management.

Backend

It uses below fields :

- status: Tracks application state (NEW, UNDER_REVIEW, APPROVED, REJECTED)
- reviewedBy: Admin UUID who processed the application
- reviewedAt: Timestamp of review
- reviewRemarks: Optional comments from admin

APIs :

- **GET /admin/loans/filter** Retrieves loan applications filtered by their statuses. Request param: List of LoanApplicationStatus values. Response: List of matching loan applications for admin review.
- **PUT /admin/loan/{loanId}/status** Updates the status of a specific loan application (approve/reject) and adds remarks. Path param: loanId (loan application ID) Body: LoanUpdateRequestDto (new status, remarks) Response: HTTP 200 OK on success.
- **GET /admin/loans** Returns all loan applications in the system for admin access. Response: List of all loan applications.

- **GET /admin/tickets** Retrieves all support tickets, auto-closing resolved ones if needed. Response: List of support tickets for admin management.
- **PUT /admin/ticket/{ticketId}** Updates the status of a support ticket to RESOLVED and adds an admin response. Path param: ticketId (support ticket ID) Body: TicketUpdationRequestDto (response message) Response: HTTP 200 OK on success.
- **GET /admin/tickets/filter** Retrieves support tickets filtered by their statuses. Requestparam: List of TicketStatus values Response: List of matching support tickets.

Business Logic :

- Only admins can access the above endpoints.
- Status transitions are validated (e.g., only NEW applications can be approved/rejected).
- Remarks are optional but can be provided on rejection.

Frontend

- Displays a list of loan applications with filters (NEW, UNDER_REVIEW, APPROVED, REJECTED).
- Action buttons for approving or rejecting applications.
- Input for optional remarks when rejecting.

Unit Testing

- JUnit tests for admin APIs and loan status transitions.
- Tests for business rules in the loan scoring and approval engine.

Module 4: EMI Calculator

Overview

EMI Calculator and Repayment Schedule Module This module enables users to calculate EMIs, preview repayment amounts, and view detailed repayment schedules for approved loans. It uses standard financial formulas and provides RESTful APIs for simulation and schedule retrieval.

Backend

Entities :

- LoanApplication: Extended to reference repayment schedules.
- RepaymentSchedule: Stores monthly principal, interest, and remaining balance for each loan.

EMI Calculation Service :

- Implements the formula: $EMI = [P \times R \times (1+R)^N] / [(1+R)^N - 1]$

- Generates a month-wise repayment plan for approved loans.

APIs :

GET /loans/{loanId}/schedule: Returns the repayment schedule for a given loan.

POST /emi/preview: Accepts amount, rate, and tenure; returns simulated EMI value.

Business Logic :

- Only approved loans can have repayment schedules.
- Accurate calculation of principal, interest, and remaining balance for each month.

Frontend

- Input fields for amount, tenure, and interest rate.
- Dynamically displays calculated EMI.
- Table view of monthly repayment details.
- Supports filtering and sorting.

Unit Testing

- JUnit tests for EMI calculation logic and schedule generation.
- Tests cover edge cases and formula accuracy.

Module 5: Customer Support Ticketing System

Overview

The **Customer Support Ticketing System** allows users to **raise support requests**, **track their current status**, and **receive updates or resolutions** from administrators.

It ensures smooth communication between customers and the support team through an **API-driven backend** and **Vue.js-powered frontend** for seamless interaction.

The system can be integrated into loan management or other customer service platforms, enabling **efficient request handling** with proper tracking, prioritization, and response management.

Backend

Entities

SupportTicket

- This is the main entity for tracking customer support requests.

Each ticket contains:

- **Ticket ID** (Unique Identifier - UUID)
- **User ID** (Reference to the user who created the ticket)
- **Loan Reference Number** (Optional – helps link the issue to a specific loan or transaction)
- **Subject** (Short title summarizing the problem)
- **Description** (Detailed explanation of the issue)
- **Status** (Enum – e.g., **OPEN**, **IN_PROGRESS**, **RESOLVED**, **CLOSED**)
- **Created Timestamp**
- **Last Updated Timestamp**
- **Admin Response** (Text or structured message from support staff)

APIs

1. POST **/support**

Purpose: Create a new support ticket for a user.

Workflow:

- The user sends a request containing **subject** and **description** (loan reference optional).
- The backend:
 - **Validates:**
 - Subject is not empty
 - Description is not empty
 - **Assigns:**
 - Unique Ticket ID (UUID)
 - Status as **OPEN**
 - Current timestamp for creation and last update
 - Saves the ticket in the database
- Returns:
 - Ticket ID
 - Current status
 - Message confirming creation

Business Logic

Only authenticated users can create/view their tickets.

Frontend

- Form for submitting new support tickets.
- Input fields: subject, description.
- Validation feedback for required fields.
- Displays a list of user's support tickets.
- Shows status and admin response if available.

Unit Testing

- JUnit tests for ticket creation, status updates, and field validation.
- Tests cover API responses and business rule enforcement.

Conclusion

The Personal Loan Application System is a robust, full-stack platform designed to streamline the complete lifecycle of personal loan management. It offers secure user authentication, an intuitive loan application workflow, administrative tools for processing and decision-making, a precise EMI calculator, and a responsive customer support ticketing system. By modularizing the architecture, the system ensures scalability, maintainability, and ease of integration with future enhancements. With a strong emphasis on RESTful API design, unit testing, and role-based access control, the application provides a reliable, user-friendly, and efficient solution for both customers and administrators in a digital lending environment.